GitHub API Integration Guide

Overview

This document covers integrating GitHub's APIs into our server to fetch user activity (commits, contributions, events). We'll go over API vs Webhooks (and why API is our only option), OAuth App vs GitHub App, REST vs GraphQL, authentication approaches (single team token vs per-user OAuth), rate limits, costs, setup instructions, and code examples. The key decision is whether we need private repo access—if not, a single Personal Access Token is the simplest path.

What We Want to Get

- **Commit history** - per repo or across all user repos
- **Contribution activity** - daily/weekly contribution counts
- **User events** - push events, PR activity, issues, etc.
- **Repository stats** - languages, commit frequency

API vs Webhooks

The choice depends on update frequency and whether you control the repos.

	API (Polling)	Webhooks
How it works	You pull data from GitHub	GitHub pushes data to you
Update speed	On-demand or scheduled	Real-time
Setup	Just need a token	Need public endpoint + webhook registered per repo
Works for any public repo	Yes	No—must own or have admin access
Cost driver	Requests you make (controllable)	Events fired (not controllable)

When to use which:

Use Case	Choice
Fetch activity from arbitrary users	API — can't install webhooks on repos you don't own
Real-time updates on your team's repos	Webhooks — you control the repos
Daily/weekly summaries	API — scheduled Lambda job
User clicks button to see stats	API — fetch on demand

For our project: We're fetching activity for arbitrary users, so **API is our only option**—you can't install webhooks on repos you don't own. Even with OAuth, you can't auto-install webhooks; the user would need to grant admin permissions, which most won't do for a progress tracker.

OAuth App vs GitHub App

Two different ways to authenticate with GitHub for user data.

	OAuth App	GitHub App
Acts as	The user who authorized it	Itself (bot) or on behalf of users
Rate limit	5,000/hour (per user token)	15,000/hour (as app)
Setup complexity	Simpler	More complex
Permissions	Broad scopes (repo, user)	Granular (pick exact permissions)
Installation	User authorizes once	Installed on specific repos/orgs
Best for	"Login with GitHub", user-specific data	Bots, CI/CD, org-wide automation

When to use which:

Use Case	Choice
Users log in to see their own activity	OAuth App

Use Case	Choice
Need access to user's private repos	OAuth App
Building a bot or automated tool	GitHub App
Need higher rate limits (15k vs 5k)	GitHub App
Want fine-grained permissions	GitHub App

github app requires more maintenance and setup work

OAuth App setup:

Register app (name, callback URL) — 2 min
Save client ID + secret
Implement token exchange endpoint
Done

GitHub App setup(JWT handling, installation management):

Register app with granular permissions — 5 min
Generate and download private key (.pem file)
Store private key securely
Implement JWT generation (sign with private key)
Exchange JWT for installation access token
Handle app installations (users "install" your app on their repos/orgs)
Done

For our project: OAuth App is the right choice because we're fetching data for individual users, it's simpler to implement, and 5,000 req/hour is plenty. GitHub App is overkill unless we need the higher rate limit or want to act independently of users. Oauth would require us to request 'repo', which needs both read and write. github app allows us to only request read.

API Options

GitHub offers two ways to fetch data—REST for simple queries, GraphQL for more complex or batched requests.

REST API

- Simple HTTP requests
- One resource per request
- Good for straightforward queries

GraphQL API

- Single request for multiple resources
- Fetch only the fields you need
- Better for contribution calendars and complex queries

Authentication: Single Token vs Per-User OAuth

A single team-owned PAT works for public data; OAuth is required only if users need access to their private repos.

Single Team Token (PAT)

- **One token** owned by a team member or service account
- Server uses it for all API calls
- Users don't need to do anything
- Works for **public data only**

Data	Works with single PAT?
Public repo commits	Yes
Public user events	Yes
Public contribution calendar	Yes
Private repo commits	No

Data	Works with single PAT?
Private contributions	No

Per-User OAuth

OAuth requires registering an app under a team member's GitHub account (or org) and implementing the auth flow—significantly more setup than PAT.

- Each user clicks "Login with GitHub" and authorizes your app
- Your app receives a token scoped to that user
- **Required for private repos and private contributions**

Bottom line: If we only need public GitHub activity, one team-owned PAT is enough. If users want their private repo data, we need OAuth.

Rate Limits & Multi-User Feasibility

Unauthenticated requests (60/hour) won't work for us since Lambda requests share AWS IPs—we need at least a PAT (5,000/hour).

Auth Method	Rate Limit	Cost
Unauthenticated	60/hour (by IP)	Free
Personal Access Token (PAT)	5,000/hour	Free
GitHub App	15,000/hour	Free

Will 60 req/hour work for multiple users?

No. Here's why:

- Our Lambda runs on AWS, so all requests come from AWS IPs (shared across many services)
- 60 requests/hour is per IP, not per user of our app
- A single user browsing their activity could use 5-10 requests
- With 10 concurrent users, we'd exhaust the limit instantly

Recommendation: Use a PAT at minimum (5,000/hour is sufficient for most use cases). A GitHub App is better if we need to act on behalf of users.

Costs

GitHub API is free at all tiers; our main costs are AWS infrastructure.

Resource	Cost
GitHub API	Free (all tiers)
AWS Lambda	~$0.20 per 1M requests
API Gateway	~$3.50 per 1M requests
Secrets Manager	$0.40/secret/month

Estimated monthly cost for MVP: < $5 (assuming < 100k requests)

Setup

Steps to get GitHub API access working in our Lambda.

1. Create a Personal Access Token

1. Go to GitHub → Settings → Developer settings → Personal access tokens → Fine-grained tokens
2. Generate new token with these permissions:
   - `public_repo` (read public repos)
   - `read:user` (read user profile data)
3. Set expiration (max 1 year, recommend 90 days)

2. Store Token in AWS

Option A: Environment Variable — Quick setup for development, but less secure and harder to rotate.

```
// wdygd_server-stack.ts
const fn = new lambda.Function(this, "BackendApiFn", {
  // ...existing config
  environment: {
    GITHUB_TOKEN: process.env.GITHUB_TOKEN // Set in CI/CD, not committed
  }
});
```

Option B: Secrets Manager — Recommended for production; secrets are encrypted, auditable, and easy to rotate.

```
import * as secretsmanager from 'aws-cdk-lib/aws-secretsmanager';

const githubSecret = secretsmanager.Secret.fromSecretNameV2(
  this, 'GitHubToken', 'github/api-token'
);
githubSecret.grantRead(fn);
```

3. OAuth Setup (Required for Private Repos)

Only needed if users want private repo data—skip this for public-only access.

Step 1: Register an OAuth App

1. Go to GitHub → Settings → Developer settings → OAuth Apps → New OAuth App
2. Fill in:
   - **Application name:** Your app name
   - **Homepage URL:** Your app URL
   - **Authorization callback URL:** `https://your-api.com/auth/github/callback`
3. Save the **Client ID** and **Client Secret**

Step 2: OAuth Flow

```
1. User clicks "Login with GitHub"
2. Redirect to: https://github.com/login/oauth/authorize?client_id=YOUR_CLIENT_ID&scope
3. User authorizes → GitHub redirects to your callback URL with a `code`
4. Exchange code for access token:
   POST https://github.com/login/oauth/access_token
   Body: { client_id, client_secret, code }
5. Store user's access token (encrypted) in your database
6. Use their token for API calls to access private data
```

Scopes for private data:

- `repo` — Full access to private repos
- `read:user` — Read user profile

4. Install Dependencies (Optional)

Octokit is GitHub's official SDK—it provides TypeScript types and pagination helpers, but adds bundle size. For 2-3 endpoints, raw `fetch` is simpler and has no dependencies.

```
cd functions/backend-entry-lambda
npm init -y
npm install @octokit/rest  # Optional: only if you want the SDK
```

Code Examples

Copy-paste examples using raw `fetch` —no external dependencies required.

Fetch User Commits (REST)

```
async function getUserCommits(owner: string, repo: string, author?: string) {
  const url = new URL(`https://api.github.com/repos/${owner}/${repo}/commits`);
  if (author) url.searchParams.set('author', author);

  const res = await fetch(url, {
    headers: {
      'Authorization': `Bearer ${process.env.GITHUB_TOKEN}`,
      'Accept': 'application/vnd.github+json',
      'X-GitHub-Api-Version': '2022-11-28'
    }
  });
  return res.json();
}
```

Fetch User Activity Feed (REST)

```
async function getUserEvents(username: string) {
  const res = await fetch(`https://api.github.com/users/${username}/events`, {
    headers: {
      'Authorization': `Bearer ${process.env.GITHUB_TOKEN}`,
      'Accept': 'application/vnd.github+json'
    }
  });
  return res.json(); // Returns last 90 days, max 300 events
}
```

Fetch Contribution Calendar (GraphQL)

```
async function getContributions(username: string) {
  const query = `
    query($username: String!) {
      user(login: $username) {
        contributionsCollection {
          totalCommitContributions
          totalPullRequestContributions
          totalIssueContributions
          contributionCalendar {
            totalContributions
            weeks {
              contributionDays {
                date
                contributionCount
              }
            }
          }
        }
      }
    }
  `;

  const res = await fetch('https://api.github.com/graphql', {
    method: 'POST',
    headers: {
      'Authorization': `Bearer ${process.env.GITHUB_TOKEN}`,
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({ query, variables: { username } })
  });
  return res.json();
}
```

Useful REST Endpoints

Quick reference for common endpoints we might use.

Data	Endpoint
User's public events	`GET /users/{username}/events/public`
Commits in a repo	`GET /repos/{owner}/{repo}/commits`
User's repos	`GET /users/{username}/repos`
Repo contributors	`GET /repos/{owner}/{repo}/contributors`
Commit activity (weekly)	`GET /repos/{owner}/{repo}/stats/commit_activity`
Code frequency	`GET /repos/{owner}/{repo}/stats/code_frequency`

Caching Strategy

GitHub data doesn't change frequently—caching reduces API calls and improves response times.

- **Contribution calendar:** Cache 1 hour
- **User events:** Cache 5-15 minutes
- **Repo stats:** Cache 1 hour (GitHub caches these server-side anyway)

Use DynamoDB or ElastiCache if we need persistent caching.

Next Steps

Action items for the team.

- [ ] Decide: Do we need private repo access? (PAT only vs PAT + OAuth)
- [ ] Create team PAT and store in Secrets Manager
- [ ] If private repos needed: Register OAuth App and implement auth flow
- [ ] Implement GitHub API endpoints in Lambda
- [ ] Add caching layer if needed
- [ ] Set up token rotation reminder (PAT expires after 90 days recommended)

Resources

Official GitHub documentation for reference.

- GitHub REST API Docs
- GitHub GraphQL Explorer
- Rate Limiting Guide
- Creating a PAT
- OAuth Apps Guide